

orthogonalization: what to tune in order to try to achieve one effect.

Chain of assumptions in ML

- fit training set well on cost function.

($\approx$ human level performance)

- fit dev set well on cost function.

- fit test set well on cost function.

- performs well in real world.

try bigger network
try different algorithms

regularizations
Bigger training set

Bigger dev set

change dev set and cost or cost function

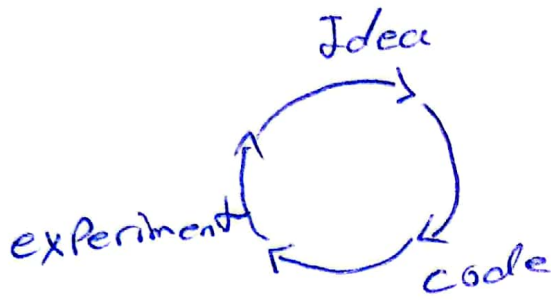
Single Number Evaluation Metric

- set up a single real number evaluation metric for your problem!

ex:

exp:

2



classifier	precision	recall
A	95%	90%
B	98%	85%

what
precision: $\rightarrow$ Percentage of recognized variables
by classifier are our variable?

recall: $\rightarrow$ what % of our variable are correctly
recognized by classifier.

which classifier is better? A or B?

solution: try F1-score to score the classifier.

$$F1\text{-score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{recall}}}$$

"Harmonic mean"
of precision
and recall

Satisficing and optimizing metrics

3

Exp:

classifier	Accuracy	Running time
A	90%	80 ms
B	92%	95 ms
C	95%	1500 ms

optimizing (pointing to Accuracy)

satisficing (pointing to Running time)

the best (pointing to B)

$$\text{cost} = \text{accuracy} - (0.5 \times \text{running time})$$

maximize accuracy

running time must be less than $< 100 \text{ ms}$

"Train/Dev/Test Distributions"

Dev stands for: Development or hold out ^{cross} validation set

- Test and Dev set must come from the same distribution.

EXP:

44

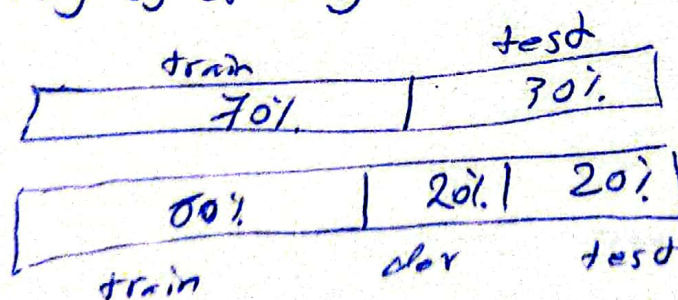
- Optimizing on dev set on loan approvals for medium income Zip codes.

Tested on low income Zip codes. X wrong

Dev and Test set had different distributions.

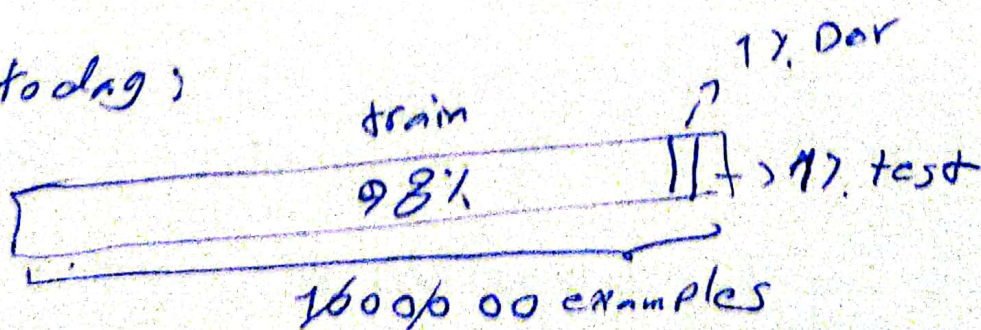
- choose a dev set and test set to reflect data you expect to get in the future and consider important to do well on.

Old way of splitting data:



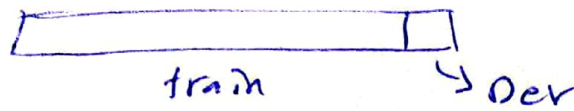
back then the size of data was smaller ^{really} than today

today:



⑤
- Set your test set to be big enough to give high confidence in the overall performance of your system.

- it might be ok if you have no test set!



when to change dev/test sets and metrics.

Exp1

metric: classification error

✗ Algorithm A: 3% error → pornographic images

✓ Algorithm B: 5% error

metric + Dev = prefer A

you / users: prefer B

so you should change your metric.

and your decision be based on the moral!

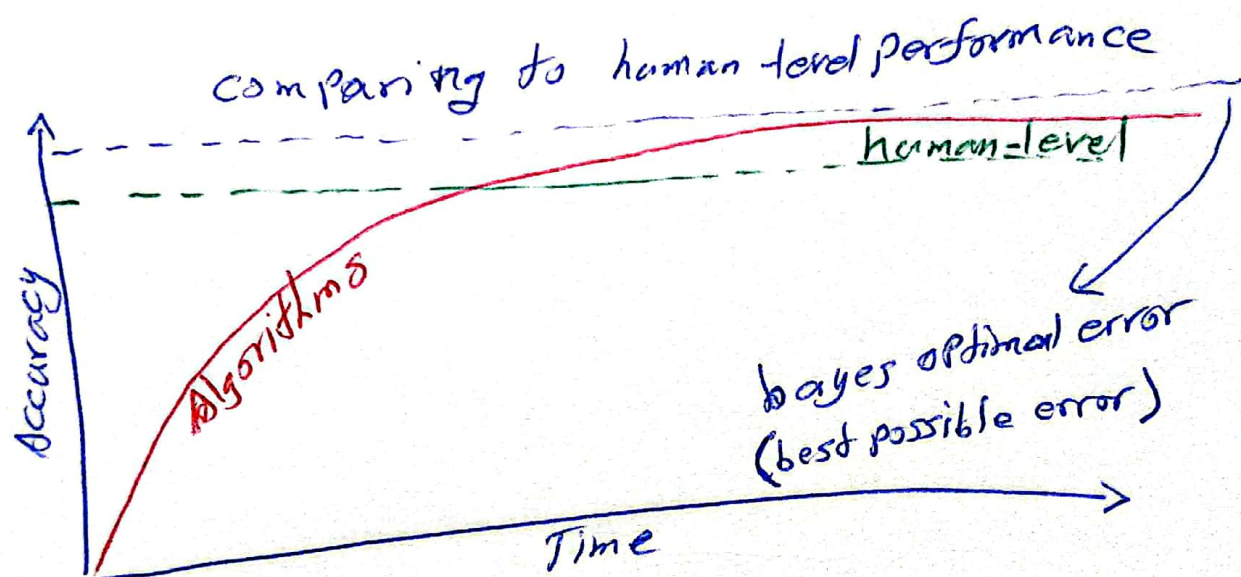
new formula to calculate error

Error: $\frac{1}{n_{dev}} \sum_{i=1}^{n_{dev}} \mathbb{I} \{ y_{pred}^{(i)} \neq y^{(i)} \}$ counts the number of miss classified examples

$y^{(i)} = 1$ if $(x^{(i)})^T w > 0$ not porn
 $w = 10$ if $(x^{(i)})$ is pornographic

⑤
- if the algorithm doing well on your metric + dev/test set does not correspond to doing well on your application, then change your metric and/or dev/test set.

why human-level performance?



- so long the performance of algorithm is lower than human-level, there ^{the used} is space to improve the performance.

solutions:

- get labeled data from humans.
- gain insight from manual error analysis: (why did a person get this right?)
- better analysis of bias/variance.

Avoidable bias

⑦

- Human's error 1%
 - Training error 8%
 - Dev error 10%
- Diagram illustrating error components:
-
- avoidable bias
- variance

Solution:

- focus on reducing bias (underfitting)

- choose a bigger network
- increase number of iterations (train longer)
- increase learning rate (α)

...

- Human error 7.5%
 - Training error 8%
 - Dev error 10%
- Diagram illustrating error components:
-
- avoidable bias (0.5)
- variance (2)

Solution:

- focus on reducing variance (overfitting)

- collect more training data.
- regularization techniques.
- decrease learning rate.

...

- Human-level error as a proxy for bayes error.

↓
this is right in certain areas like computer vision,

understanding human-level performance

medical image classification example:

Supposes

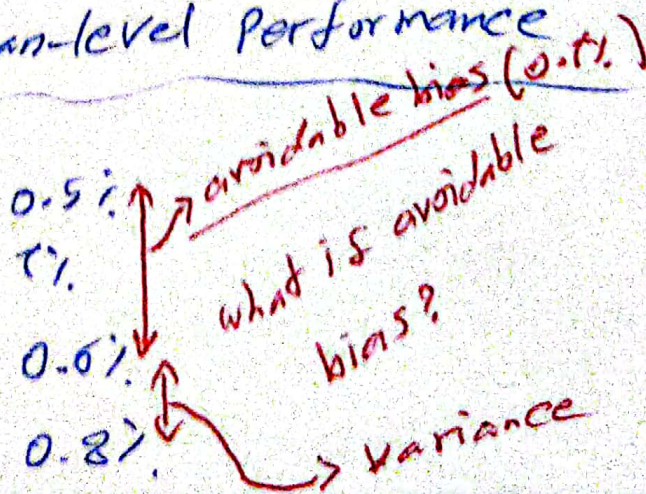
- Typical human 3% error
- Typical doctor 1% error
- experienced doctor 0.7% error
- Team of experienced doctor 0.5% error
- Bayes error $\leq 0.5\%$

what is human-level error?

- Human-level error as a proxy for bayes error

surpassing human-level performance

- Team of humans
- one human
- Training error
- Dev error



- Team of humans 0.5 %
- one human 1 %
- Training error 0.3 %
- Dev error 0.4 %

what is the avoidable bias ?!

not enough information

- problems where ML significantly surpasses human-level performance.

exps:

- online advertising
- product recommendations
- logistics (predicting transit time)
- loan approvals

Structured data

not natural perception problems

Human is very good in natural perception
Problems like computer vision, natural language processing

- ~~speech~~ speech recognition

- Some image recognition tasks.

10

- Medical tasks.

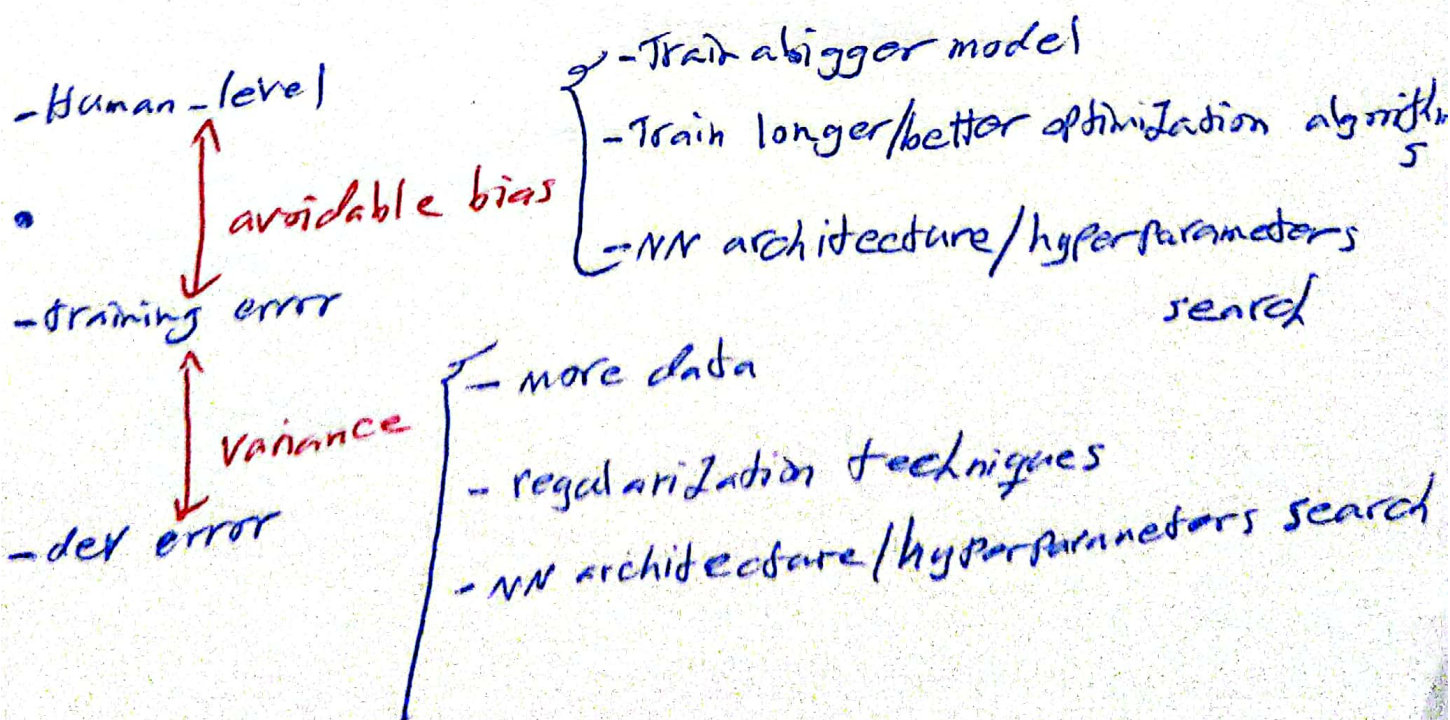
Improving your model performance

- Two fundamental assumptions of supervised learning

1. You can fit the training set pretty well.

2. The training set performance generalizes pretty well to the dev/test set.

= Reducing (avoidable) bias and variance.



Error Analysis

99

EXP: a model to classify cats } 90% accuracy
model performance } means
10% error

Error analysis

- Get ~100 mislabeled dev set examples.

- count up how many are dogs.

→ the model recognizes
- if 5% of mislabeled are dogs. Some dogs as cat.
it means if you solve the dog problem,

your model performance will increase by only 10.5
means 79.5 or the model error will decrease
to 19.5.

So ~~the~~ the manual analysis (error analysis) which you
did, gives you an intuition about the situation
of the model.

cleaning up incorrectly labeled data

- if there are some incorrectly labeled data in your
training or dev set,

- if in training sets if the error is randomly it's ok, you
don't have to spend too much time to fix them

- NN are robust to random errors in the training set (12) but they are less robust to systematic errors.
- if in dev or test sets analyse them and find out more about it and if it is worth to fix them.
- apply same process to your dev and test sets to make sure they continue to come from the same distribution
- consider examining examples your algorithm got right as well as ones it got wrong.
- Train and dev/test data, after corrections, may now come from different distributions.

Project Process

- Set up dev/test set and metric
- build initial system quickly and train it.
- use bias/variance analysis and error analysis to prioritize next steps.

Interpretable

- if your data ~~comes~~ comes from different distributions
Put them together and shuffle it and then divide it
into train/dev/test sets.

- another option (recommended) is to put all data
which comes from a different distribution and is more
important than other distributions into test and
dev sets.

Bias/Variance on mismatched training and dev/test sets.

- Human-level error : 14%
- Training set error : 0.7%
 - avoidable bias
- Train-dev set error (same distribution as train set) : 10%
 - variance
- dev error : 12%
 - data mismatch
- Test error : 13%
 - degree of overfitting to dev set.

Addressing data mismatch

→ carry out manual error analysis to try to understand difference between training and dev/test sets.

- make training data more similar, or collect more data similar to dev/test sets.

(exp) synthesis of new data

 +  = 
"the quick brown fox jumps over the lazy dog."
car noise
synthesized in-car audio

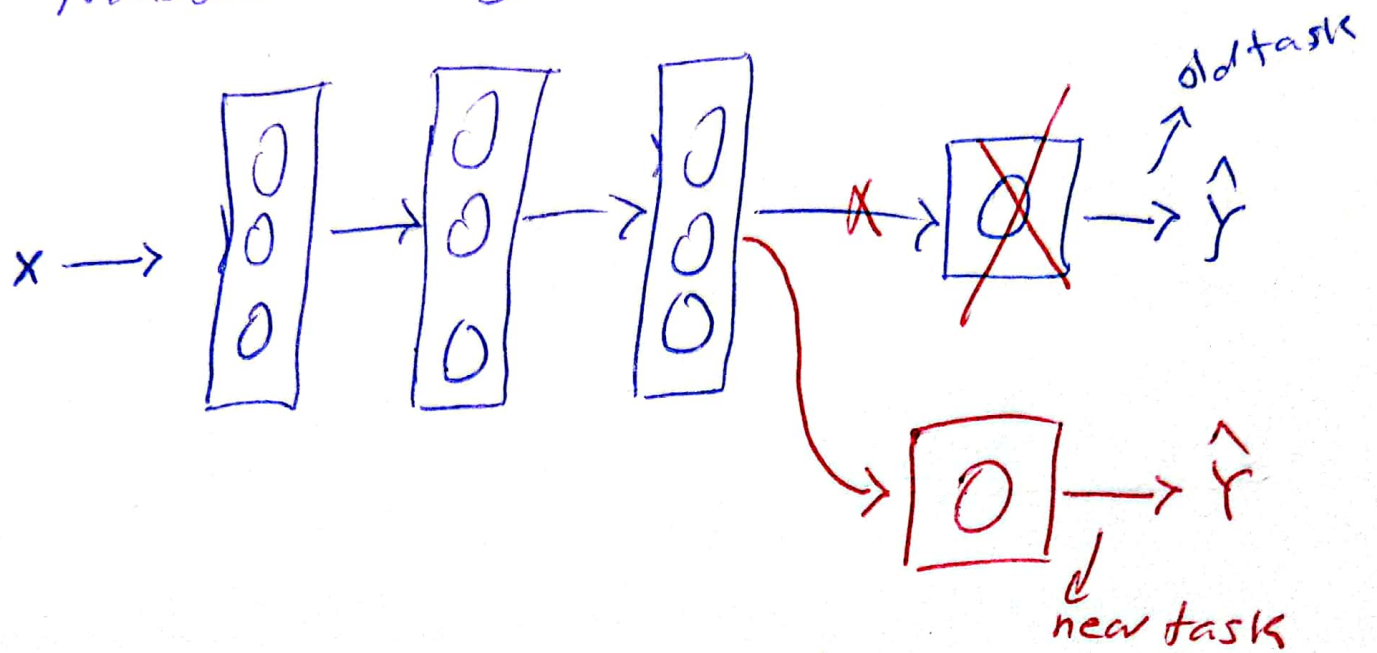
Transfer learning

learned
- use the knowledge of in a model to do a different task.

Exp: a model which is trained on images (image recognition task)
use this model and adapt (transfer) it to another task such as "radiology diagnoses"

Transfer learning:

15



after training the model on the first task (in this example image recognition) we can use this model ~~to~~ and train it on a new data set (radiology diagnosis) in this case we just reinitialize the parameters of the last part layer and try to adapt the model on the new dataset.

— if the new dataset (new task) is small then you reinitialize the parameters of the last layer and retrain the model just the ~~last~~ this parameters (other parameters in other layers are frozen) and if you have enough data you can retrain all the parameters belongs to ~~the~~ the entire network.

- if you retrain the entire network, the (16) ~~22~~

first phase of training is called "pre-training"

and the second part of training - train the model on the second data set, is called

"fine-tuning"

- in some cases you can add more than one layers to the network and retrain it.

- when ~~it~~ does "transferring ^{learning} ~~data~~" make sense?

(when you have a small set of new data and the old model ~~is~~ was trained on a relatively big data set.

→ Transfer from Task A to task B.

- Task A and B have the same input X.

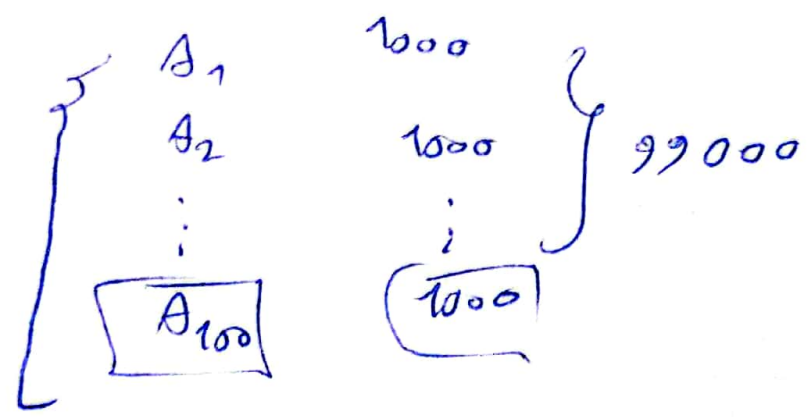
(both images, audio or...)

- you have lot more data for task A than task B.

- low level features from A could be helpful for learning B.

when does multi-task learning make sense?

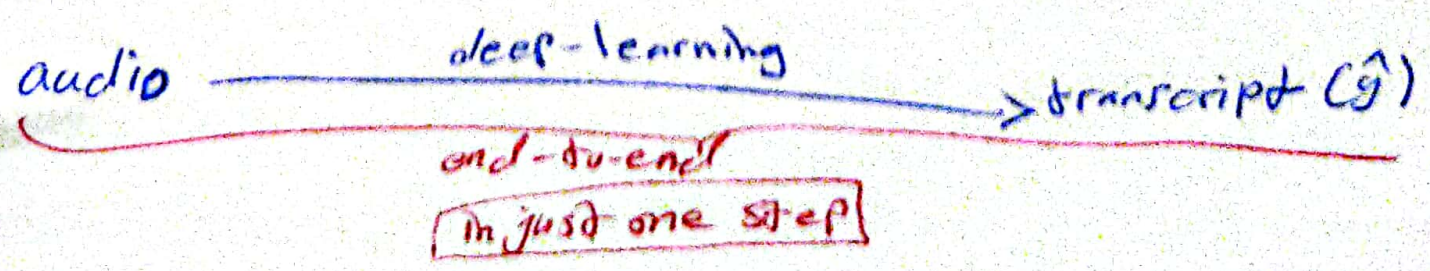
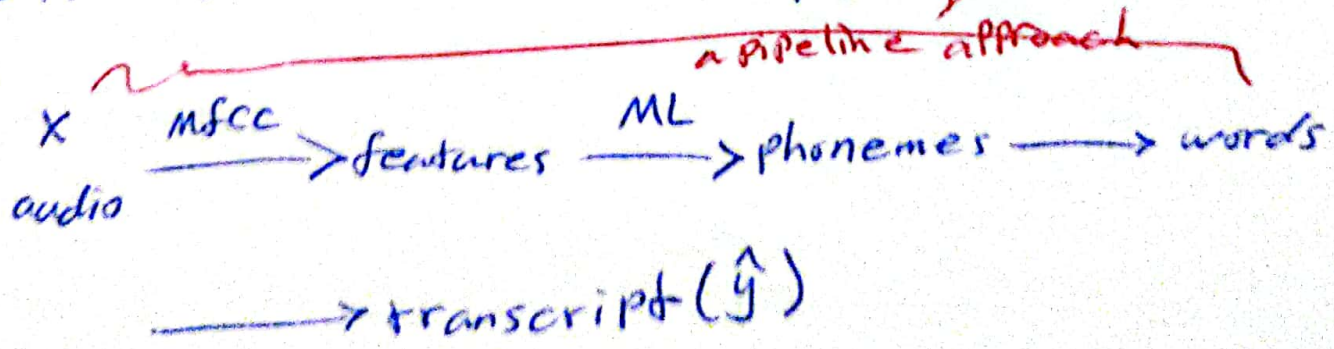
- Training on a set of tasks that could benefit from having shared lower-level features.
- usually, amount of data you have for each task is quite similar.



- can train a big enough neural network to do well on all the tasks.

End-to-End deep-learning

EXP: speech recognition example, ^{classic}



Exp:

autonomous driving examples
 $x^{(i)}$

Pedestrian
 cars
 stop signs
 traffic lights
 :

$y^{(i)}$
 0
 1
 1
 0
 :

an image which contains multiple objects and the image is labeled with multiple labels each for one different object.

$$\text{loss} = \frac{1}{n} \sum_{i=1}^m \sum_{j=1}^q L(\hat{y}_j^{(i)}, y_j^{(i)})$$

logistic loss

unlike softmax regression in this case ~~each~~
 each image has more than one label.

- if you retrain the entire network, the (10) 

first phase of training is called "pre-training"

and the second part of training - train the model on the second data set, is called

"fine-tuning"

- in some cases you can add more than one layers to the network and retrain it.

- when ~~may~~ does "transferring ^{learning} ~~data~~" make sense?

when you have a small set of new data and the old model ~~to~~ was trained on a relatively big data set.

→ Transfer from Task A to task B.

- Task A and B have the same input X.

↳ both images, audio or...

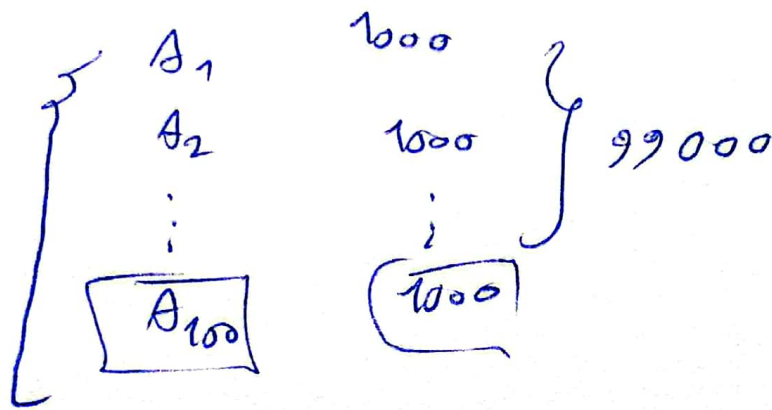
- You have lot more data for task A than task B.

- low level features from A could be helpful for learning B.

when does multi-task learning make sense?

(18)

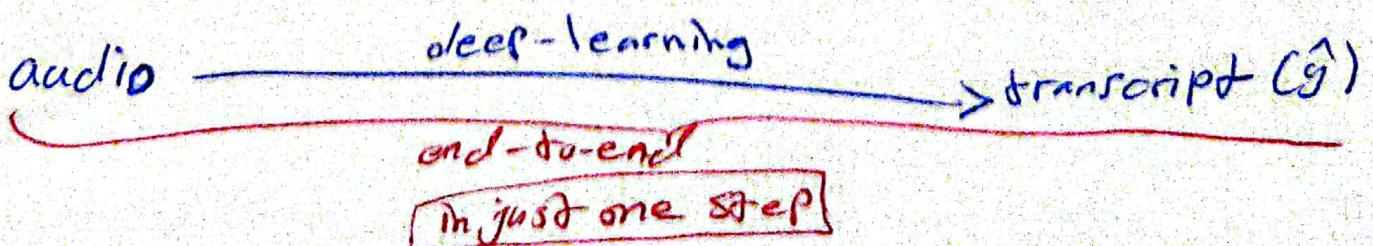
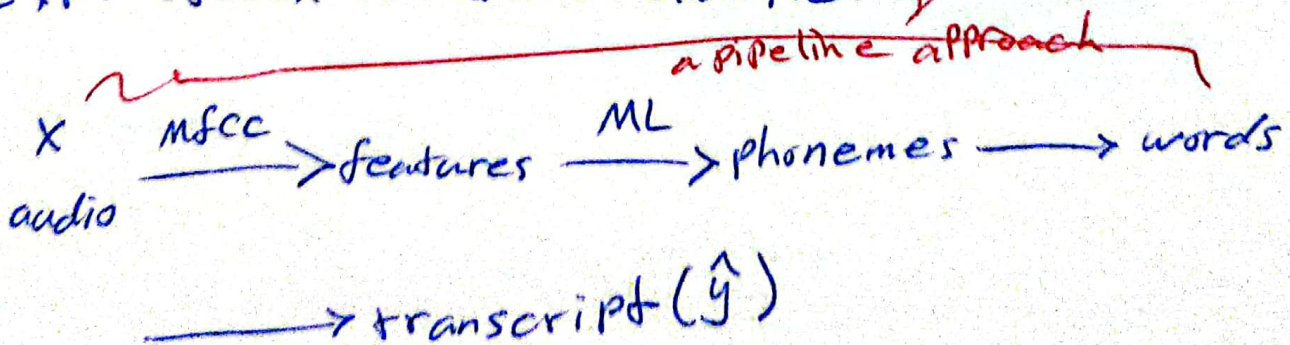
- Training on a set of tasks that could benefit from having shared lower-level features.
- usually, amount of data you have for each task is quite similar.



- can train a big enough neural network to do well on all the tasks.

End-to-End deep-learning

Exp: speech recognition example, ^{classic}



machine translation

classical pipeline approach

19

English → text analysis → . . . → french

English → french

end-to-end approach

The end